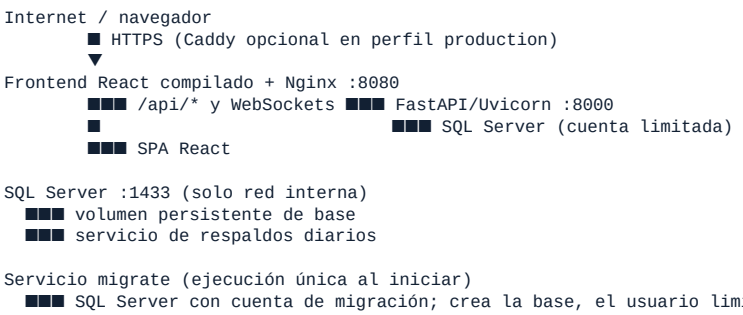
Despliegue completo con Docker Compose — UTA SUMO ROBOTICS CUP

Esta guía levanta el frontend React, la API FastAPI, SQL Server, las migraciones, copias de seguridad locales y, cuando hay un dominio, HTTPS automático. Los comprobantes siguen en un volumen persistente y los reglamentos se conservan en SQL Server. API y base de datos no publican puertos en Internet; el frontend es la entrada pública.

Arquitectura



El proceso WebSocket vive en memoria de FastAPI; por ahora se debe desplegar una sola réplica de `api`. El perfil de HTTPS usa Caddy, que también reenvía WebSockets.

Requisitos del servidor

• Docker Engine y Docker Compose v2; en Windows, Docker Desktop con backend WSL2.

• Linux x86-64 (Intel/AMD) para el contenedor oficial de SQL Server; el soporte del contenedor de SQL Server no cubre ARM.

• Para SQL Server, asigna al menos 2 GB de RAM al entorno Docker; 4 GB o más es más razonable para API, frontend y DB juntos.

• En despliegue público: dominio propio, DNS A/AAAA hacia el servidor y puertos TCP 80/443 disponibles para HTTPS. No publiques 1433.

• Elige una edición de SQL Server con licencia apropiada para el uso. `Developer` sirve para desarrollo/pruebas, no para producción. `Standard` requiere licencia; evalúa `Express` solo si sus límites bastan.

1. Preparar la configuración

Desde la raíz del repositorio:

```
cp .env.example .env
```

En PowerShell:

```
Copy-Item .env.example .env
```

Edita `.env` y reemplaza todos los marcadores `CAMBIAR_...` con secretos aleatorios distintos. No subas `.env` al repositorio ni compartas la contraseña de `sa`.

Genera `SECRET_KEY` y contraseñas con un gestor de secretos o Python:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

`PII_ENCRYPTION_KEY` requiere exactamente 32 bytes codificados como Base64 URL-safe:

```
python -c "import secrets,base64; print(base64.urlsafe_b64encode(secrets.token_bytes(32)).decode().rstrip('='))"
```

Guarda esa clave de forma segura y conserva exactamente la misma al reiniciar o actualizar el servicio. Sin ella no se podrán descifrar los datos sensibles ya guardados.

Variables que deben quedar coherentes:

Variable	Valor requerido
DB_SERVER	db dentro de Compose.
DB_NAME	UTA_SUMO_ROBOTICS_CUP, o un nombre SQL válido.
DB_USERNAME / DB_PASSWORD	Cuenta de la API; Compose crea el login y le da lectura/escritura solo en la base de la app. Usa uta_api como usuario.
MIGRATION_DB_USERNAME	sa; se usa solamente durante la migración y el alta del primer login de aplicación.
MSSQL_SA_PASSWORD	Contraseña fuerte de sa; SQL Server la exige al inicializarse. Compose la suministra al migrador, nunca al contenedor API.
DB_EDITION	Desarrollo: Developer. Producción: edición licenciada elegida para el servidor.
SECRET_KEY	Secreto aleatorio de al menos 32 caracteres para firmar JWT.
PII_ENCRYPTION_KEY	Clave Base64 URL-safe de 32 bytes, estable y respaldada fuera del servidor.
GOOGLE_CLIENT_ID	Client ID web de Google; se compila en React y también lo valida el backend. Debe ser el mismo en ambos lados.
VITE_API_BASE_URL	Origen completo que publica el frontend. Local: http://localhost:8080. Producción: https://tu-dominio. Las rutas /api/... y /ws/... se agregan en el código.
PUBLIC_DOMAIN	Host público, por ejemplo torneo.midominio.ec, para el perfil de producción.
CORS_ORIGINS	En producción, el origen exacto https://torneo.midominio.ec; en local, http://localhost:8080. No uses *.

En Google Cloud Console, registra como origen JavaScript el origen que verán los usuarios (por ejemplo, https://torneo.midominio.ec; en desarrollo, http://localhost:8080). No se configura URI de redirección para el botón GIS que utiliza el frontend.

SMTP es opcional. Si no se configura, los correos se guardan en EmailOutbox para revisión/reintento administrativo; para correo real configura SMTP_HOST, SMTP_PORT, SMTP_USERNAME, SMTP_PASSWORD y SMTP_FROM_EMAIL.

2. Iniciar en modo local

Usa DB_EDITION=Developer en .env solo para desarrollo/pruebas:

```
docker compose up --build -d
```

En Windows PowerShell el mismo comando funciona. Espera a que web y api aparezcan healthy:

```
docker compose ps
docker compose logs -f db migrate api web backup
```

Abre http://localhost:8080. El endpoint de salud se consume dentro de la red de Compose; la API no queda publicada directamente.

3. Primera cuenta administrativa

Por seguridad no se distribuye una contraseña de administrador fija ni se crea una cuenta privilegiada universal al construir la imagen. Después de iniciar los servicios, crea el operador desde una terminal interactiva:

```
docker compose run --rm migrate python scripts/provision_operator_user.py
```

Escribe correo, nombre, apellido, rol ADMIN y una contraseña segura de 15 o más caracteres dos veces. La contraseña no se imprime ni se guarda en texto plano. Usa el correo y la contraseña que acabas de definir para entrar al panel.

Este paso se hace una vez por cada instalación. La cuenta se almacena en la base persistente. No uses cuentas demo en un servidor público.

4. Publicar con HTTPS y dominio

• Configura el DNS A (y, si aplica, AAAA) de PUBLIC_DOMAIN hacia la IP pública del servidor.

• En .env, define PUBLIC_DOMAIN=tu-dominio, CORS_ORIGINS=https://tu-dominio, y configura el mismo origen en Google Cloud Console.

• Abre TCP 80 y 443 en firewall/proveedor; Caddy necesita llegar desde Internet para emitir/renovar certificados.

• Cambia `DB_EDITION` por una edición licenciada adecuada. No uses Developer en producción.

• Inicia el perfil de publicación:

```
docker compose --profile production up --build -d
```

El sitio queda en `https://tu-dominio`. Caddy administra certificados TLS y conserva su estado en volúmenes. El puerto local `8080` permanece ligado a `127.0.0.1` para diagnóstico desde el propio servidor; los puertos SQL/API no se publican al host.

5. Migraciones y actualizaciones

Las migraciones se ejecutan automáticamente antes de iniciar FastAPI. Cada script se registra con checksum en `dbo.SchemaMigrations`; los ya aplicados no se vuelven a ejecutar. Si se agregan migraciones en el código, se reconstruye y se levanta Compose:

```
docker compose up --build -d
```

Para ejecutar el migrador manualmente:

```
docker compose run --rm migrate python scripts/migrate.py
```

El migrador conecta como `sa`, pero la API usa `DB_USERNAME` y solo recibe `db_datareader`/`db_datawriter`. No cambies la cuenta de la API por `sa`.

6. Estado, registros y pruebas rápidas

```
docker compose ps
docker compose logs --tail=200 migrate api web
curl http://localhost:8080/
```

En Windows PowerShell:

```
Invoke-WebRequest http://localhost:8080/ -UseBasicParsing
docker compose ps
docker compose logs --tail=200 migrate api web
```

El frontend usa rutas de React en modo SPA. Las solicitudes `/api/...` se reenvían a FastAPI; las rutas WebSocket `/api/...` y `/ws/...` se conservan con upgrade y timeout extendido.

7. Datos persistentes y respaldos

• `sqlserver-data`: archivos de SQL Server.

• `app-storage`: comprobantes y otros archivos administrados por la API.

• `sqlserver-backups`: copia completa comprimida de SQL Server; el servicio `backup` ejecuta un respaldo diario por defecto y retiene 14 días. Ajusta `BACKUP_INTERVAL_SECONDS` y `BACKUP_RETENTION_DAYS` si lo necesitas.

• `caddy-data` y `caddy-config`: certificados y estado de Caddy.

Verifica los respaldos:

```
docker compose logs --tail=100 backup
docker volume ls
```

Los respaldos en un volumen del mismo servidor no protegen contra pérdida del servidor o del disco. Copia periódicamente el contenido de `sqlserver-backups` a otra máquina o almacenamiento de objetos y prueba la restauración antes de operar el evento.

`docker compose down` conserva los volúmenes. No uses `docker compose down -v` en un despliegue con datos: elimina la base y los archivos persistentes.

8. Parar y reanudar

```
docker compose stop
docker compose start
```

Para eliminar contenedores manteniendo datos:

```
docker compose down
```

Limitaciones operativas conocidas

• Mantén una réplica de FastAPI por el administrador WebSocket en memoria. Para escalar horizontalmente se requiere pub/sub compartido (por ejemplo Redis) y manejo de sesiones compartidas.

• No se incluye un proveedor SMTP; se conecta el servidor SMTP que la organización configure.

• SQL Server Linux en contenedor requiere arquitectura x86-64. Confirma arquitectura, memoria y licencia con el proveedor del hosting antes de elegir la máquina.

• Caddy automatiza TLS si el dominio resuelve al servidor y 80/443 están accesibles; sin dominio se usa HTTP local.

• Esta configuración organiza el despliegue, pero antes de producción se deben guardar secretos en el gestor de secretos del hosting y copiar los respaldos fuera de la máquina.

Referencias oficiales

• SQL Server en contenedores Linux: inicio rápido

• Variables de entorno de SQL Server Linux

• Orden de inicio y dependencias saludables de Compose